CTRL Stage — Verification Plan

UVM Verification Environment | Version 0.1 (draft)

Project: ctrl_uvm_environment | October 2026

1. Overview

Module: the Control (CTRL) stage of the IPU, the first stage of the pipeline MULT → ACC → AAQ → STORE. Spec: stage-control (ipu-emulator repository). CTRL owns the program counter, the double-buffered instruction memory with a one-entry instruction register (inst $), the 16 × 32-bit CR and LR register files, three LR ALU lanes, the branch resolver, and the four dispatch buses plus the XMEM read-address port.

The environment executes **programs** (sequences of VLIW words) on the DUT and verifies, cycle by cycle, that CTRL:

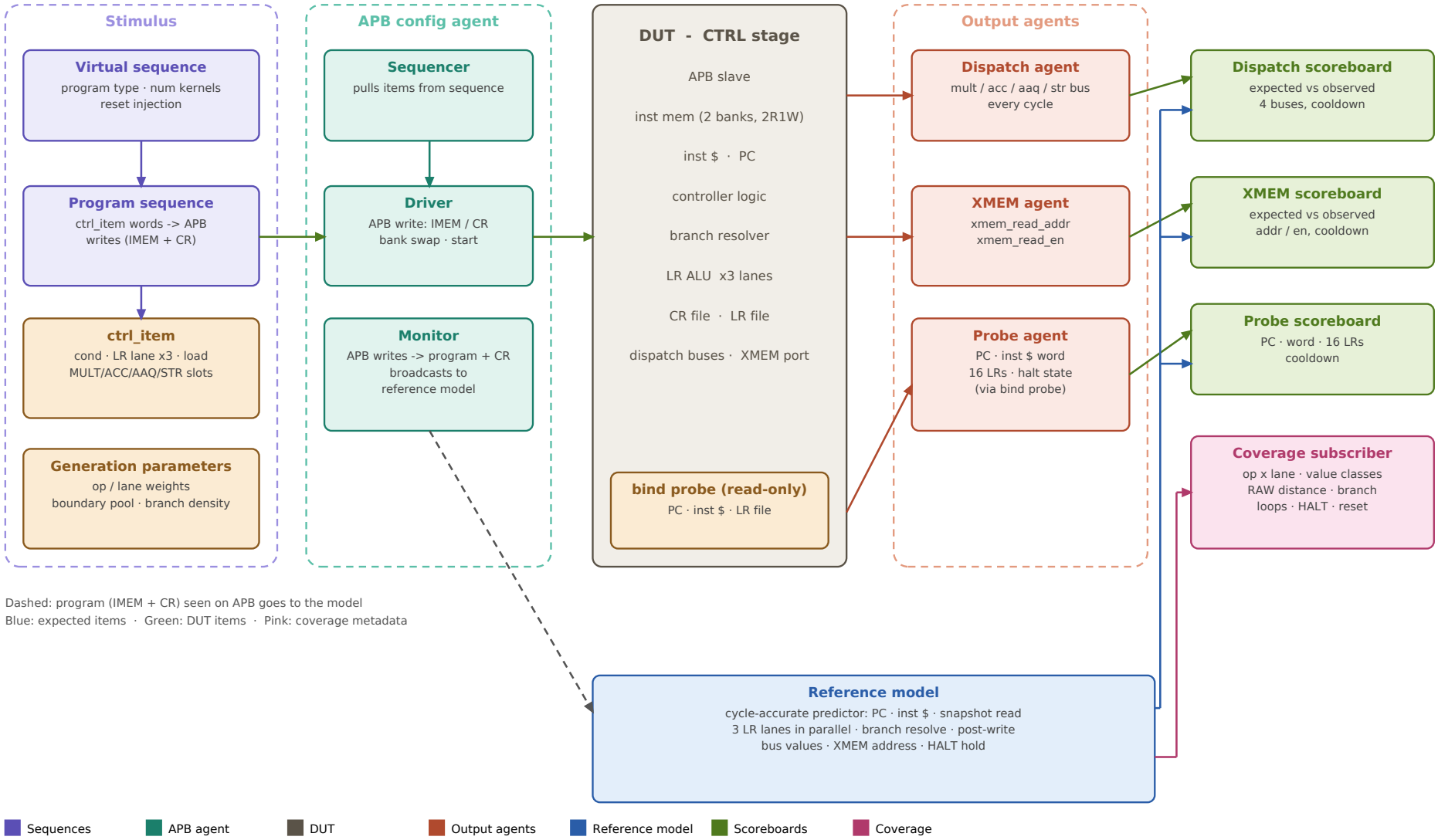
- executes the three LR lanes in parallel, each lane reading the register values from the **start of the cycle**;
- resolves branches and fetches both candidate words in parallel, with **no bubble**, and lets only the chosen word have an effect;
- stops after HALT, finishing the other slots of the HALT word exactly once;
- drives the four dispatch buses and the XMEM address with this cycle's **post-LR-write** operand values.

Scope of this plan: the commands only (LR slot × 3 lanes, COND slot, fetch / PC, load address, dispatch). APB configuration behavior and the memory-wait / stall behavior are **out of scope** for now: APB is used only to load programs, and memory wait is not yet defined in the spec (section 9).

Verification approach: stateful constrained-random program generation; a cycle-accurate reference model fed from the same APB traffic as the DUT; three scoreboards that compare expected and observed items per cycle; each expected item has a cooldown timeout; functional coverage closes the value, hazard, branch and HALT classes.

2. Environment architecture

Each scoreboard receives expected items from the reference model and observed items from the matching output agent, and compares them. The coverage subscriber receives per-cycle metadata from the reference model.



3. Agents

The environment reuses the generic DV agent (`dv_agent`), parameterised by data width. The same class drives and monitors by toggling `is_master`.

3.1 APB config agent (master)

Sequencer / driver. Takes a program (list of VLIW words plus the CR contents) and performs the APB writes into the inactive IMEM bank and the inactive CR bank, then triggers the bank swap and the kernel start. The driver can randomise wait cycles between APB writes.

Monitor. Watches the APB interface, rebuilds the program and the CR contents from the observed writes and broadcasts them to the reference model through an analysis port. The model therefore never depends on the generator, only on what was really written.

Note: how a kernel is started and how completion is signalled is not visible on the CTRL ports (Open point O7); the agent hides it behind `start_kernel()`.

3.2 Output agents (slave, monitor-only)

Three monitor-only agents sample every clock cycle. Each item carries the cycle number.

Agent	Signals monitored	Item produced
Dispatch agent	mult_vliw_bus, acc_vliw_bus, aaq_vliw_bus, str_vliw_bus	One item per cycle: the four bus values
XMEM agent	xmem_read_en, xmem_read_addr	One item per cycle: enable and (when enabled) address
Probe agent	bind probe: PC, inst $ word, LR0–LR15, halt-hold flag	One item per cycle: PC, executed word, 16 LR values after the cycle, halt state

Bind probe. PC, inst $ and the LR file are internal to CTRL. A read-only `bind` module exposes them to the probe agent; nothing is ever driven through it. Without the probe, LR errors would be seen only when a later bus or load happens to use the wrong value.

4. Sequences and items

4.1 ctrl_item — one VLIW word

A `ctrl_item` is one decoded VLIW word. A program is an array of items plus the CR2–CR14 contents. Legal-by-construction constraints are inside the item.

Field	Type	Description
cond	rand op + operands	NOP, BEQ, BNE, BLT.S, BGE.S, BR, HALT. reg1 / reg2 / reg: any LR or CR. label: target inside the active bank.
lr[3]	rand op + operands	Per lane: NOP, SET, ADD, SUB, INC.MODP2, INC, DEC, ADDB, ADDBI with dest, sources, imm / k. Constraint: no two lanes write the same LR; ADDB / ADDBI count as writing LR(n) and LR(n+1).
load	rand	NOP or {base CR index, offset LR index}. No opcode.
mult / acc / aaq / str	rand fields	Random slot fields or NOP. CTRL only passes them on, together with resolved CR / LR operands.
k	rand [3:0]	INC.MODP2 mask width, semantic range 1–9 (encoded k−1).
imm	rand [7:0]	INC / DEC: unsigned. ADDBI: signed byte (0–255 and −128–127 are the same bit pattern).

4.2 ctrl_program_sequence

Builds one program and sends it to the APB driver. It is **stateful**: it keeps a shadow copy of LR and CR (the same predictor code as the reference model) so it can choose the next word to reach a wanted value class — for example to bring an LR next to a wrap point, or bytes of an LRD pair next to 0 or 255. Every program ends with HALT. Branches are forward-only, except loops, which come from a counter template (set counter, body, DEC + BNE) so that every program terminates.

4.3 ctrl_virtual_sequence — test scenario controller

The virtual sequence runs four phases per kernel:

- **Phase 1 — Reset and CR pool.** Apply reset. Fill CR2–CR14 once per kernel from a weighted pool: about 30% boundary values (0, ±1, 0x7FFFFFFF, 0x80000000, 0xFFFFFFFF, 2^k edges, byte-saturation patterns), the rest random.
- **Phase 2 — Program.** Select a `program_type` (table below) and generate the program.
- **Phase 3 — Load and run.** Load through APB, start, wait for HALT, then wait one cooldown period so late items arrive.
- **Phase 4 — Follow-up.** Optionally inject a reset at a random moment, or run the next kernel back-to-back with a different program.

program_type	Description	What it stresses
LR_ONLY	Straight-line words, 1–3 random LR lanes, random loads and stage slots, no branches.	Every LR op, operand types, value classes, lane parallelism.
HAZARD	Directed builders: reader and writer in the same word (any lane pair); producer–consumer at distance 1…N; branch whose compared register is written in the same word or the word before.	Start-of-cycle reads, visibility of an LR write in the next word, post-write buses.
BRANCHY	Random forward branches of every opcode, taken and not taken, back-to-back and chained; BR with CR / LR targets; poison words (LR write, load, stage slots) in both candidate words.	Dual fetch, no bubble, no side effect from the non-chosen word.
LOOPS	Counter loops, nested up to 3 levels, trip counts including 0 and 1; INC.MODP2 pointer walks.	Loop timing and counters, circular-buffer wrap.
HALT_EDGE	HALT at the first and last entry, reached by fall-through / label / BR; HALT in the non-chosen candidate; HALT word with LR writes, load and stage slots.	Fetch stop, single execution of the HALT word, quiet afterwards.
RESET_INJECT	Any of the above with reset asserted at a random cycle (mid-run, branch cycle, after HALT), random pulse length.	PC to 0, dropped LR writes, scoreboard flush.
MIXED	Long fully random legal programs with all slots filled, 1000+ words.	Everything together against the model.

4.4 Generation parameters

A configuration object (`ctrl_generation_parameters`) holds all weights. It is randomised once per test and distributed through config_db.

Parameter group	Controls
program_type weights	Relative probability of each program type.
Lane and op weights	Probability each lane is used (0–3 lanes per word) and of each of the 8 LR ops; alias probability (dest = source).
Boundary pool weight	Share of CR values and steered operands taken from boundary classes (default 30%); steer probability.
Branch density	Share of words that are branches; opcode weights; probability of operands equal, ±1 apart or on the sign boundary.
Loop limits	Max trip count, nesting depth, probability of trip count 0 / 1.
Load density	None / sparse / every word; probability of an offset LR written in the same word.
HALT / reset	HALT at an edge, HALT in the non-chosen candidate, reset probability and pulse length.
Scoreboard	COOLDOWN_CYCLES (and per-channel overrides), START_LATENCY.

5. Reference model

The reference model (`ctrl_reference_model`) is a **cycle-accurate** predictor. It receives the program and the CR contents from the APB monitor, executes one word per cycle with the rules of the spec, and emits one expected item per cycle to each scoreboard and metadata to the coverage subscriber. It may wrap the ipu-emulator CTRL model if that model is cycle-accurate (to be confirmed).

5.1 State

PC, `inst $`, `LR[0..15]` (32 bit), the active CR bank (CR0 = 0, CR1 = 1), `fetch_stopped`. Reset: PC = 0, pending writes dropped.

5.2 One cycle

```
snap      = LR (values committed at end of previous cycle)       // CTRL's own reads use this
for lane in 0..2:  result[lane] = LR_OP(lane, snap)           // all lanes read snap, in parallel
taken     = COND(inst$.cond, snap)                             // branch operands also use snap
LR_new    = snap with result[0..2] applied                     // no two lanes write the same LR
buses     = pack(slot fields, CR / LR_new values)              // dispatch sees post-write values
xmem_addr = CR[load.base] + LR_new[load.offset];  xmem_en = (load != NOP)
if cond == HALT:  fetch_stopped = 1                            // other slots of this word still run
else:             PC = taken ? target : PC + 1                 // target = label, or reg[6:0] for BR
// only the chosen word is executed; the other fetched candidate has no effect on LR, buses or load port
emit expected {PC, word, LR_new, buses, xmem}
```

5.3 Operation semantics

Op	Result	Notes
SET dest src5	src5[4]=0: CR[src5[3:0]]; src5[4]=1: sign-extended 4-bit imm	−8…+7
ADD / SUB	(a ± b)[31:0]	b = LR or CR; wrap, no saturation
INC.MODP2 dst step k	(dst + step) & ((1<<k) − 1)	k 1–9; mask applied to the sum
INC / DEC dest imm	(dest ± imm)[31:0]	imm unsigned 8-bit
ADDB / ADDBI LRDn	each of 8 bytes of LR(n+1):LR(n): clamp(byte + signed val, 0, 255)	ADDB uses src_b[7:0]; no carry between bytes; byte order: Open point O5
BEQ / BNE	taken if reg1 == / != reg2	32-bit compare
BLT.S / BGE.S	taken if signed(reg1) < / >= signed(reg2)	signed compare
BR reg	always taken, target reg[6:0]	PC width 7 bit
HALT	stop fetching; PC and inst $ hold	other slots of the word run once

5.4 Rules the model must get right

- **Snapshot vs post-write:** an LR write of word N is invisible to CTRL's own reads (lane sources, branch operands) in word N and visible from word N+1; the dispatch buses and the XMEM address in word N already see it.
- **No bubble:** the correct next word executes in the next cycle for taken and not-taken branches.
- **Non-chosen candidate:** no LR write, no load, no bus item from the word that was fetched but not chosen.
- **HALT:** after the HALT word the model emits no item and holds until restart or reset.

6. Scoreboards

Three instances of a generic scoreboard (`ctrl_scoreboard #(ITEM)`) compare reference-model items with DUT items. All three share the same mechanism.

6.1 Comparison mechanism

Each scoreboard has two analysis ports: `env_port` (from the reference model) and `dut_port` (from the output agent). Items are pushed onto two queues. Whenever both queues are non-empty, the front items are popped and compared field by field. Because CTRL has fixed latency and no bubble, the n-th expected item must be matched by the n-th DUT item, and it must arrive in the cycle the model predicts.

Instance	Item	What is compared
Dispatch scoreboard	4 bus values per cycle	mult / acc / aaq / str bus: slot fields and resolved post-LR-write operand values
XMEM scoreboard	enable + address per cycle	xmem_read_en; xmem_read_addr when enabled
Probe scoreboard	PC, word, 16 LRs per cycle	Executed PC and word, all 16 LR values after the cycle, halt state

6.2 Cooldown (timeout)

Every expected item that arrives from the reference model starts a **cooldown counter**, counted in clock cycles. If the matching DUT item has not arrived when the counter passes `COOLDOWN_CYCLES`, the scoreboard raises **UVM_FATAL** (`SB_TIMEOUT`) and prints the expected item. It is fatal because nothing can be compared any more.

Event	Result
DUT item arrives within the cooldown at the predicted cycle, content equal	Items removed from the queues, counter cleared
Content differs	UVM_ERROR with the field-by-field difference
Arrives inside the cooldown but at a wrong cycle	UVM_ERROR (timing); the content is still compared
No DUT item before the cooldown expires	**UVM_FATAL** SB_TIMEOUT, channel and expected item printed
DUT item arrives while no expected item is pending	UVM_ERROR (unexpected item)
Activity after HALT	UVM_ERROR (no expected item exists, no cooldown started)
Expected items still pending at end of test	UVM_FATAL SB_PENDING. The test waits COOLDOWN_CYCLES after HALT before ending.
Reset	Queues and counters flushed; no counter runs while rst = 1; the first item's counter starts at reset release
Memory wait (spec not defined)	Check disabled. When defined, the counter must pause or the cooldown extend during a legal stall.

There is one counter per channel (the four buses, XMEM port, PC trace, LR probe), so the message names the channel that stopped. Default `COOLDOWN_CYCLES` = 16, at least `START_LATENCY` + 1; it only decides how fast the fatal fires, because exact arrival is checked separately. The largest observed arrival delay per channel is printed in the final report to tune the value.

7. Functional coverage

The coverage subscriber (`ctrl_coverage`) receives a coverage item from the reference model every cycle, so a class is counted when it actually executed, not when it was generated. The following coverpoints and crosses must be closed.

7.1 Instruction-level coverpoints

Coverpoint	What is measured	Required bins
LR op × lane	Which op ran in which lane; dest; operand type	8 ops × 3 lanes; dest 0–15; LR / CR / imm
Value class (per op)	Operand / result class of each LR op	zero, ±1, max / min, wrap edge, saturation edge; INC / DEC imm 0 / 127 / 128 / 255; SET imm min / max / −1
INC.MODP2 k	Mask width and wrap	k = 1…9 × wrapped / not wrapped × step sign; dst above mask
ADDB / ADDBI saturation	Byte behavior	none / high / low / mixed saturation × 8 pairs; src_b with non-zero upper bits; imm both spellings
Lanes per word	Number of active lanes and pairs	0 / 1 / 2 / 3 lanes; 1 / 2 / 3 ADDB pairs in one word
Branch	Opcode and outcome	BEQ / BNE / BLT.S / BGE.S / BR × taken / not taken × operand relation (equal, ±1, sign boundary, other) × operand type
Load	Load density and address	none / sparse / every word; offset written in same word; address past XMEM_ADDR_W; negative offset

7.2 Scenario-level coverpoints

Coverpoint	What is measured	Required bins
RAW distance	Distance between an LR write and its first reader	1, 2, 3, 4+ × reader (LR lane, branch, bus, load offset)
Same-word dependency	A lane reads a register another lane writes in the same word	6 lane pairs × writer op
Branch shape	Where the branch goes	target 0 / PC / PC+1 / last entry / other; back-to-back; poison in the non-chosen word
Loop	Loop trip count and depth	trip 0 / 1 / 2 / many × depth 1–3
HALT	Where and how HALT is reached	first / last / other × fall-through / label / BR; HALT in non-chosen candidate
Reset	When reset was applied	idle / mid-run / branch cycle / after HALT × short / long pulse
Arrival delay	Cycles between expected and observed item	only the exact-latency bin in a passing run; cooldown FATAL and unexpected-item ERROR each seen once in a negative test

7.3 Required cross coverage

Cross	Axes	Purpose
op × lane × value class	LR op, lane 0–2, value class	Every lane handles every op and every boundary class, not only lane 0.
RAW distance × producer op × reader	distance, producer op, reader kind	Every op's result is usable in the very next word by every reader.
branch op × taken × operand relation	opcode, outcome, relation	Signed compare and equality at the boundaries for every branch.

8. Key parameters

Parameter	Value	Description
REG_WIDTH	32 bit	Width of every CR and LR (diagram in the spec says 20 bit; see O1)
LR / LRD / CR registers	16 / 8 / 16	LR0–15; LRD0, LRD2 … LRD14 pair aliases; CR0–15 per bank (CR0 = 0, CR1 = 1)
LR lanes	3	Independent LR sub-instructions per word
LR ops / COND ops	8 / 6	SET, ADD, SUB, INC.MODP2, INC, DEC, ADDB, ADDBI / BEQ, BNE, BLT.S, BGE.S, BR, HALT
IMEM	2 banks × 128	Double-buffered; PC width 7 bit
INC.MODP2 k	1–9	Encoded k − 1 on 4 bits
INC / DEC imm	8 bit unsigned	Shared field with ADDBI (signed)
XMEM_ADDR_W, bus widths	implementation	Taken from the RTL
COOLDOWN_CYCLES	16 (tunable)	Scoreboard timeout per expected item
START_LATENCY	TBD	Cycles from start event to first dispatch (O7)
Clock period	TBD	Not stated in the spec

9. Open points

The spec has gaps and inconsistencies that decide what the model and scoreboard may expect. To be closed with the designer before the model is finalised.

#	Open point	Affects
O1	CR / LR width: 20 bit in the block diagram, 32 bit in the parameters. This plan assumes 32.	All wrap and overflow classes
O2	Branch label: absolute address (§6.3) or PC-relative offset (§10.2)?	Branch target in the model
O3	'Up to three LR writes per cycle' but ADDB / ADDBI write two registers each; what if pairs overlap other lanes? Behavior when two lanes write the same LR is not defined.	Generator legality constraints
O4	LR reset value and encoding of an empty LR lane are not stated.	Probe scoreboard after reset; empty-lane stimulus
O5	Byte order inside LRDn (which LR holds bytes 0–3).	ADDB / ADDBI model
O6	Memory wait: the spec mentions a 'bubble' and a section that does not exist, while stating there are no interlocks. Behavior of PC, LR commit and buses during a wait is undefined.	Cooldown must pause; a held word must not apply INC / DEC twice
O7	Start / done of a kernel is not a CTRL port; START_LATENCY unknown.	APB agent, first-item cooldown
O8	INC.MODP2 encodings for k above 9; PC+1 past the last entry; BR value ≥ 128.	Illegal-program behavior (excluded by the generator)
O9	Whether the ipu-emulator CTRL model is cycle-accurate and reusable as the reference model.	Section 5